

FUNDAMENTOS DE LA IAC Y EL CLOUD

EMPEZANDO POR LO BÁSICO

INFRAESTRUCTURA DE TI

*Conjunto de componentes físicos, lógicos y de red que actúan como la **base tecnológica** de una solución digital.*

- **Cómputo:** Servidores físicos o virtuales (CPU/RAM).
- **Almacenamiento :** Discos duros, NAS, SAN, Bases de datos.
- **Redes:** El "sistema circulatorio" (Switches, Routers, Firewalls, Cableado/Fibra).

APROVISIONAMIENTO DE INFRAESTRUCTURA

Es el proceso de preparar y equipar la red y el hardware para que estén listos para ejecutar software.

Es importante distinguir entre:

- **Planificación:** Diseñar la arquitectura donde se ejecutará el software
 - Diseño de la arquitectura de despliegue
- **Aprovisionamiento:** "Crear" la máquina
 - Comprar el servidor o crear la instancia en AWS
- **Configuración:** Instalar las librerías, el OS, parches y la aplicación.

TAREAS DURANTE PLANIFICACIÓN Y APROVISIONAMIENTO

- Configurar CPU/RAM
- Asignar Espacio.
- Conectarse a la red y configurar reglas de conexión.
- Instalar sistema operativo y software necesario.

Entre otras...

EJEMPLOS DE ESTRATEGIAS DE PROVISIÓN DE INFRAESTRUCTURA

CASO 1: LA CONSOLA EN CASA

Tienes que ir a la tienda, comprar la consola comprar la TV y los cables.

Ventajas

- Tienes control total
- Baja latencia (no hay lag por internet).

Desventajas

- Inversión inicial alta (CAPEX)
- Si se rompe el ventilador, tú lo arreglas. Si quieres mejores gráficos años después, tienes que comprar una consola nueva (Escalado difícil).

Tú eres el responsable

CASO 2: XBOX CLOUD, GEFORCE NOW

Eliges y pagas la suscripción del servicio de gaming

Ventajas

- Tú solo necesitas una pantalla y un mando "tonto". El juego corre en un servidor de Google o Microsoft.
- Si sale un juego nuevo muy potente, tú no haces nada.
 - **Escalado:** El proveedor actualiza sus servidores

Desventajas

- Si se cae internet, no juegas.
 - **Dependencia**

El proveedor es responsable

¿QUÉ ES UN CPD?

*Las siglas CPD hacen referencia a Centro de Procesamiento de Datos
Data Center*

Instalación física centralizada donde se alojan sistemas informáticos críticos.

Componentes vitales:

- **Potencia eléctrica:** Doble acometida (si falla una, entra la otra) y generadores diésel de respaldo.
- **Refrigeración:** El enemigo nº1 de los servidores es el calor.
- **Conectividad:** Múltiples proveedores de fibra óptica.

Problemática de los CPDs: refrigeración y energía

EJEMPLOS INTERESANTES RETOS CPDS

PROYECTOS SUBMARINOS Y ESPACIALES

Sumergir los CPDs

- **La Lógica:**
 - El fondo del mar está frío de forma natural (refrigeración gratis) y cerca de la costa vive el 50% de la población mundial (baja latencia).
- **Resultado:**
 - Descubrieron que los servidores fallaban *menos* bajo el agua porque no había humanos golpeándolos ni oxígeno que causará corrosión.
- Fuentes: **El Confidencial**, **Microsoft**

CPDs Espaciales

- **La lógica:**
 - Temperatura: más baja fuera de la atmósfera.
 - Energía solar 24/7
 - SpaceX está interesada
- **Resultado:**
 - TBA 😊
- Fuentes: **Business Insider**, **Reuters**, **Google R&D**

CPDS EN REGIONES FRÍAS

- Meta en Luleå (Suecia): Situado a las puertas del Círculo Polar Ártico.
- Google en Hamina (Finlandia): Esta instalación utiliza una antigua fábrica de papel
- CPDs subterráneos en refugios y minas

La Lógica:

- **Refrigeración**
 - Usar el frío polar: abren las ventanas y usan ventiladores gigantes para meter aire helado (Free Cooling).
 - Usar el agua de mar ártica: para enfriar el hardware de manera pasiva.
 - La estabilidad térmica subterránea combinada con el clima exterior optimiza el rendimiento energético al máximo.
- **Energía:**
 - La zona tiene presas hidroeléctricas masivas, por lo que la energía es 100% renovable y muy barata.

APROVECHANDO LOS "RESIDUOS"

- **"Deep Green"** (Reino Unido): Calentando piscinas municipales.
- **Calefacción Urbana** (Finlandia): Recuperando hasta el 90% del calor que generan operadores como Telia o Equinix
- **Meta en Odense** (Dinamarca): Canaliza el calor residual para abastecer la calefacción de hogares.

La Lógica:

- Los servidores generan calor.
- **Simbiosis:** El "centro de datos" cede su calor gratis para calentar el agua.
- Ejemplo perfecto de economía circular.

MODELOS DE ARQUITECTURAS DE IT

- On-premise
- En el cloud
- Híbridas

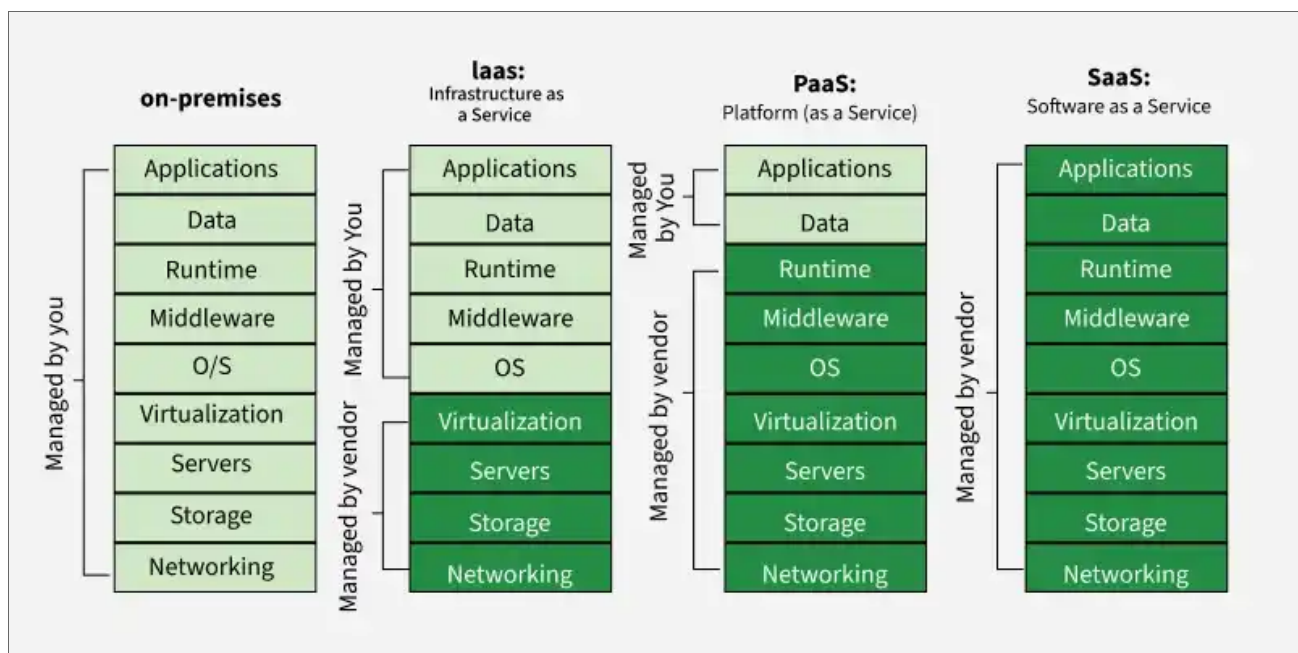
ARQUITECTURAS ON-PREMISE

*La empresa es la dueña absoluta de su tecnología, instalando y gestionando la infraestructura en sus **propias instalaciones físicas**.*

- Ventajas:
 - **Control total** sobre hardware, red y datos.
 - **Personalización extrema** integración con sistemas legados.
 - **Requisitos regulatorios** que obligan a mantener datos in-house.
- Inconvenientes:
 - **Alto coste** inicial en compra de hardware y licencias.
 - **Responsabilidad completa** de seguridad, parches, backups, disponibilidad.
 - **Escalado lento**: comprar, instalar, cablear.

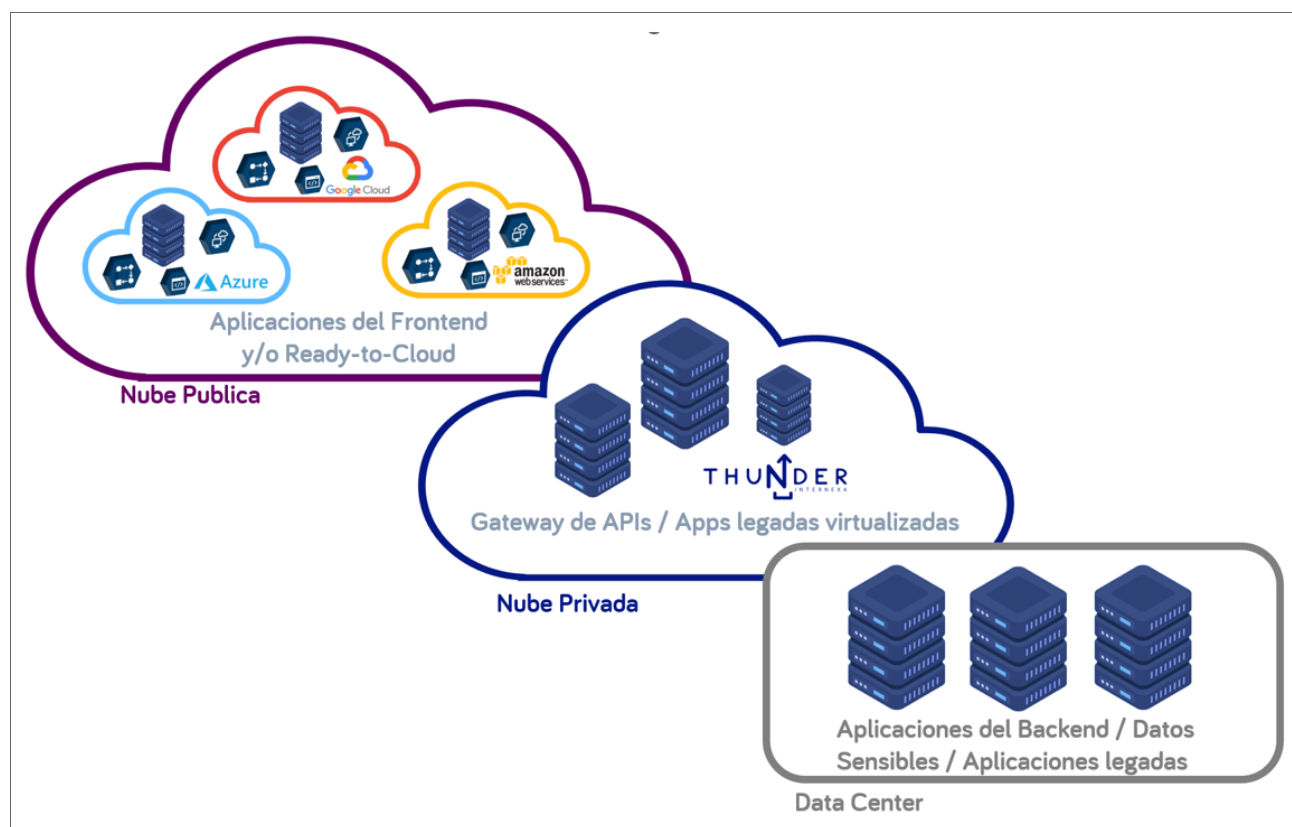
ARQUITECTURAS EN CLOUD (MODELOS DE SERVICIO Y DESPLIEGUE)

La infraestructura vive en centros de datos de un proveedor; se accede a recursos de forma remota y bajo demanda, pagando por uso.



- Modelos de servicio (muy por encima):
 - IaaS: máquinas virtuales, redes, almacenamiento.
 - PaaS: plataforma para desplegar aplicaciones, ocultando parte de la infraestructura.
 - SaaS: aplicación final consumida como servicio.

MODELOS DE DESPLIEGUE



- **Nube pública**
 - Infraestructura en el proveedor, compartida entre clientes.
- **Nube privada**
 - Infraestructura dedicada para cada organización
 - Gestionada con modelo “tipo nube”.
- **Nube híbrida**
 - on-premise + nube

ARQUITECTURAS HÍBRIDAS

Combina recursos locales con servicios de nube pública

- Motivaciones típicas:
 - Cumplimiento y residencia de datos
 - Mantener ciertos datos on-premise, explotar servicios gestionados en la nube.
 - Latencia y proximidad al edge
 - Procesar cerca de planta industrial, hospitales, retail, etc.
 - Modernización gradual de legados
 - Extraer funcionalidades a microservicios en cloud manteniendo bases de datos locales.
- Integración operativa y de red entre datacenter propio y nube pública
 - Diferencia fundamental frente a multi-cloud

COMPONENTES CLAVE EN UNA ARQUITECTURA HÍBRIDA

- Conectividad:
 - VPN site-to-site, enlaces dedicados para tráfico privado y latencia predecible.
 - Azure ExpressRoute, AWS Direct Connect
- Identidad y seguridad:
 - Integración de directorios locales con Azure AD / IAM, SSO, MFA
 - Segmentación de red,
 - Centralización de seguridad y políticas cero confianza.
- Gestión y observabilidad:
 - Panel de control único para inventario, políticas, monitorización y
 - Automatización sobre recursos on-premise y cloud

CASOS DE IMPLEMENTACIONES HÍBRIDAS

- **La Ley (Cumplimiento):**
 - Obligación de que los datos de los clientes estén en un servidor físico en España (On-premise)
 - Quiero usar la IA del proveedor de Cloud para analizar tendencias de gasto.
- **El "Cloud Bursting" (El efecto rebote)**
 - Servidor local aguanta 100 usuarios.
 - El exceso de tráfico se "desvía" automáticamente a la Nube para que la web no explote.
- **La mudanza eterna (Legacy)**
 - Tengo una aplicación de hace 20 años que no se puede mover al Cloud
 - La dejo donde está, pero la conecto con servicios modernos en la nube.

FUNDAMENTOS DE LA INFRAESTRUCTURA COMO CÓDIGO

¿QUÉ ES LO QUE VAMOS A APRENDER?

1. Tipos de gestión de infraestructura IT.
2. Beneficios clave de la IaC.
3. Principios de la IaC
4. Enfoques de la IaC.

INTRODUCCIÓN HISTÓRICA



PET VS CATTLE

- Tradicionalmente la gestión de máquinas ha sido como cuidar una mascota
 - Ejemplos UCM: heimdal (Eduroam), librero (Entregas prácticas / exámenes), etc.
 - Los Administradores de sistemas se conectaban para configurar de manera específica la máquina.
- Problemas:
 - No es un proceso escalable.
 - La configuración manual es propensa a errores
 - Tensión entre Sistemas / Operaciones y Desarrollo
- Solución:
 - Aunque nos gusten las mascotas, tenemos que tratar a los servidores / VMs como ganado para que el trabajo sea sostenible y reproducible.

La Infraestructura como código (IaC) es una aproximación que facilita este proceso.

INFRAESTRUCTURA COMO CÓDIGO

Gestión y aprovisionamiento de infraestructura de TI (servidores, redes, bases de datos) mediante archivos de definición legibles por máquina en lugar de configuración manual

- Problemas que resuelve
 - Deriva de configuración
 - Podemos tener los scripts en un repositorio de código (e.g. git)
 - Podemos aplicar los procesos de revisión de calidad y pruebas que en el código de aplicación
 - Falta de reproducibilidad
 - La misma plantilla puede ser aplicada para crear diferentes entornos: integración, pruebas / QA, producción

PRINCIPIOS FUNDAMENTALES:

1. Idempotencia en IaC
2. Inmutabilidad en IaC

IDEMPOTENCIA EN IAC

Garantizar que el resultado sea siempre el mismo, sin importar cuántas veces se ejecute.

- Importancia en IaC
 - Fiabilidad
 - Las operaciones se pueden reintentar sin causar efectos secundarios no deseados.
 - Consistencia:
 - Se garantizan que el estado final del sistema sea consistente y predecible,
 - Independientemente del número de veces que se haya realizado la operación.
 - Simplicidad:
 - Facilitan su razonamiento y depuración por su comportamiento es determinista y predecible.

¿CÓMO NOS AFECTA LA IDEMPOTENCIA?

*Las herramientas que utilicemos para soportar la IaC deben asegurarse de **no aplicar los cambios si el sistema está en el estado deseado.***

- Varias opciones de implementación:
 1. Se aplica sobre la infraestructura existente
 2. Se crea un nuevo entorno completamente nuevo aplicando los cambios
 - Se descarta el entorno ya existente una vez probado el nuevo.

Opción 1 la más habitual y 2 si te lo puedes permitir

INMUTABILIDAD EN IAC

Definición: No modificar la infraestructura existente, sino reemplazarla.

- ¿Por qué es relevante?
 - Nos permite minimizar (o eliminar) inconsistencias en la configuración
 - Las inconsistencias se introducen por configuraciones manuales en los recursos
 - Problemas introducidos al automatizar operaciones que son difíciles de depurar para todos los casos.
- Solución
 - No permitimos acceso / configuración manual a la infraestructura.
 - Si se necesitan recursos TIC diferentes
 - Desplegamos una plantilla nueva de infraestructura
 - Migramos datos
 - Desplegamos aplicación
 - Comprobamos que la aplicación funciona
 - Eliminamos los recursos TIC antiguos
- Terraform soporta tanto modificación de recursos existentes como recreación de recursos
 - En caso de que no pueda modificarlo => lo recrea.

ENFOQUES DE IAC

1. Declarativo
2. Imperativo

ENFOQUE DECLARATIVO

*Definimos **qué** infraestructura queremos.*

- Definimos el estado final deseado:
 - Qué recursos queremos (e.g. una VM, una BD)
 - Qué características tienen estos recursos (e.g. RAM, CPU, durabilidad)
- La herramienta **calcula** las acciones necesarias para llegar al estado deseado a partir del estado inicial.
- Soluciones declarativas: Terraform / OpenTofu, CloudFormation.

ENFOQUE IMPERATIVO

*Definimos **cómo** lograr la infraestructura que queremos*

- **Definimos los pasos** para alcanzar el estado deseado:
 - Definimos las acciones concretas a realizar.
 - Las ordenamos de manera precisa para obtener el resultado deseado.
- Soluciones imperativas: script bash, Ansible, Puppet, Salt, Chef, etc.

DECLARATIVO VS IMPERATIVO

- No: **Declarativo + Imperativo**
- Declarativo:
 - Se usa para aprovisionar la infraestructura global
- Imperativo:
 - Se usa para configurar de manera específica algunos recursos
 - Tradicionalmente VMs donde desplegar el aplicativo
 - Generación de GOLDEN images

SISTEMAS DE IAC EN LA ACTUALIDAD

- Soluciones específicas de un proveedor
 - CLI, Clicks-Ops
 - IaC del proveedor
 - AWS: AWS CloudFormation
 - GCP: GoogleCloud Deployment Manager
 - Microsoft: Azure Resource Manager
- Soluciones multi-proveedor
 - Terraform
 - OpenTofu
 - Pulumi
 - Librería de Python
 - Aprovechar experiencia del programador de Python

QUÉ ES LO QUE HEMOS APRENDIDO

Resume presentation

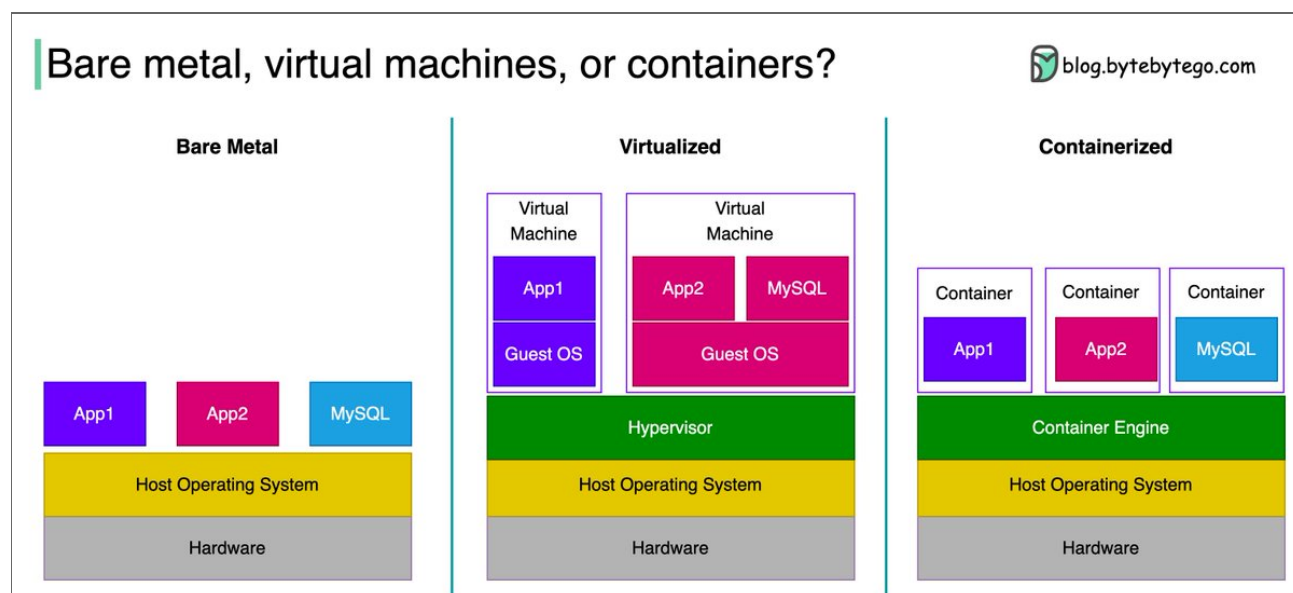
1. A Diferenciar entre la gestión de infraestructura tradicional y la gestión mediante código.
2. Articular los beneficios clave de la IaC, como la velocidad, la consistencia y la reducción de riesgos.
3. Comprender y explicar los principios de idempotencia e inmutabilidad.
4. Distinguir entre un enfoque declarativo y uno imperativo, identificando las herramientas principales de cada uno.

FUNDAMENTOS DEL CLOUD

¿QUÉ ES LO QUE VAMOS A APRENDER?

1. Diferentes modelos de servicio y despliegue de la nube
2. Componentes básicos de infraestructura en la nube.
3. Responsabilidad compartida en la nube.

METAL VS MÁQUINAS VIRTUALES VS CONTAINERS



MODELOS DE SERVICIO (ALTO NIVEL)

- IaaS
 - Máquinas virtuales, redes, almacenamiento.
- PaaS
 - Plataforma para desplegar aplicaciones, ocultando parte de la infraestructura.
- SaaS
 - Aplicación final consumida como servicio.

IAAS

AWS EC2

- **Tú haces:**
 - Instalas el software, configuras el firewall, gestionas las actualizaciones del sistema operativo y escalas la capacidad.
- **AWS hace:**
 - Mantiene los servidores físicos, la electricidad y el cableado.

PAAS

AWS Elastic Beanstalk

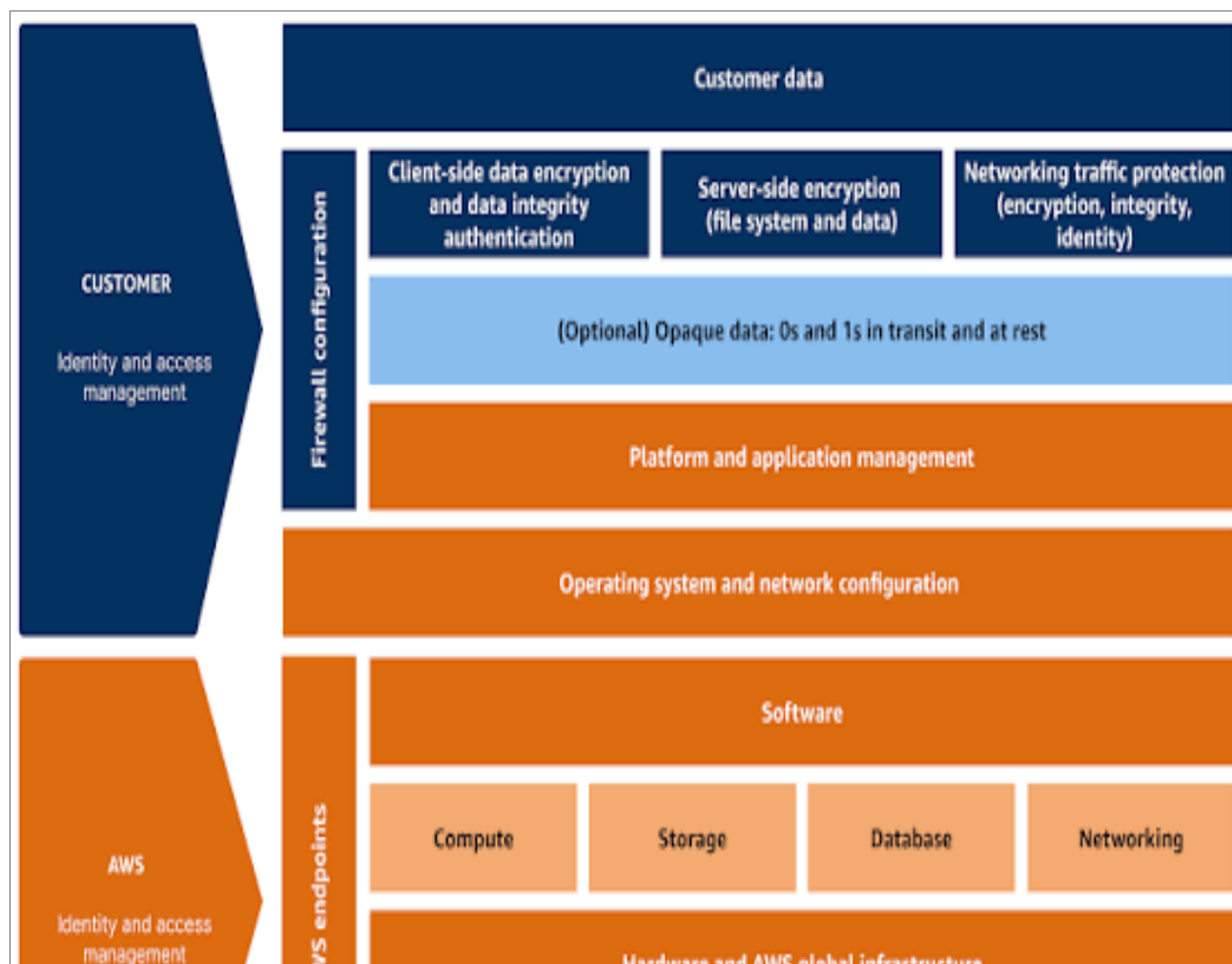
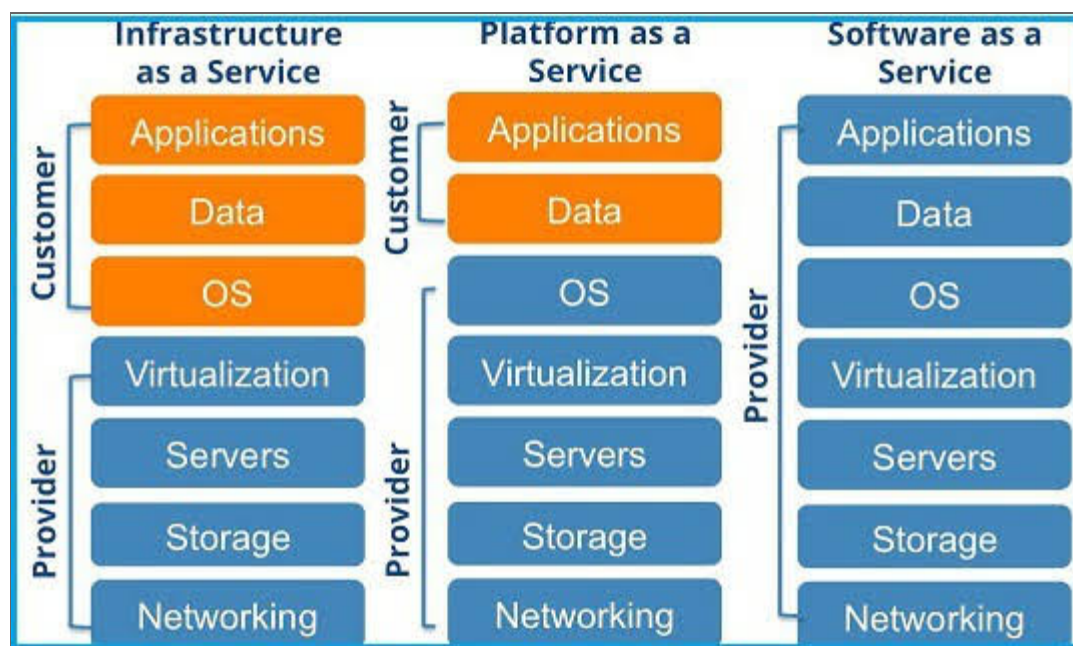
- **Tú haces:**
 - Subir el código.
 - **Si usas Python:** Subes un `.zip` con tus archivos `.py` y un archivo `requirements.txt` (donde dices qué librerías necesitas).
 - **Si usas Node.js:** Subes tu carpeta con los archivos `.js` y el `package.json`.
 - **Si usas Java:** Subes el archivo `.jar` o `.war` ya compilado.
- **AWS hace:**
 - Gestiona el sistema operativo, el servidor web (Apache/Nginx), los parches de seguridad del software base y el autoescalado.

SAAS

AWS Lambdas

- **Tú haces:**
 - Escribes la lógica del código (ej. "si suben una foto, cámbiale el tamaño").
- **AWS hace:**
 - TODO lo demás. Solo te cobra por los milisegundos que tu código estuvo corriendo.

MODELO DE RESPONSABILIDAD COMPARTIDA





PROVEEDORES DE LA NUBE

- Proveedores Principales (Hiperscalares)
 - AWS, Azure, Google Cloud Platform, Oracle
- Otros proveedores:
 - OVH
 - Hetzner
 - DigitalOcean

AWS: INTRODUCCIÓN Y SERVICIOS

- AWS sigue siendo el "gorila de 800 libras" en la habitación.
- Microsoft Azure y Google Cloud crecen rápido gracias a la IA, AWS mantiene la corona de la infraestructura.
- **Cuota de Mercado**
 - **Dominio Global:**
 - AWS ostenta aproximadamente el **31% de la cuota de mercado** de infraestructura cloud (IaaS/PaaS).
 - **El "Big Three":**
 - Azure (25%) y Google Cloud (11%)
 - Las tres controlan más del 65% del gasto mundial en la nube.
 - **Rentabilidad:**
 - Es el motor financiero de Amazon, generando más del 50% de sus beneficios operativos totales.

CASOS DE ÉXITO DE AWS

- AWS es la columna vertebral de internet.
- **Entretenimiento**
 - **Netflix** (corre casi al 100% en AWS), **Disney+** y **Epic Games** (Fortnite).
- **Tecnología y RRSS**
 - **Meta** (Facebook/Instagram), **Airbnb**, **Slack** y **Pinterest**.
- **Industria y Finanzas:**
 - **BMW**, **Volkswagen** y la **NASA**.
- **IA de vanguardia:**
 - Recientemente han cerrado acuerdos masivos con **OpenAI** (para ciertas cargas de trabajo), **Anthropic** y **Perplexity**.



AWS Global Infrastructure

Regions, Availability Zones & Edge Locations Explained



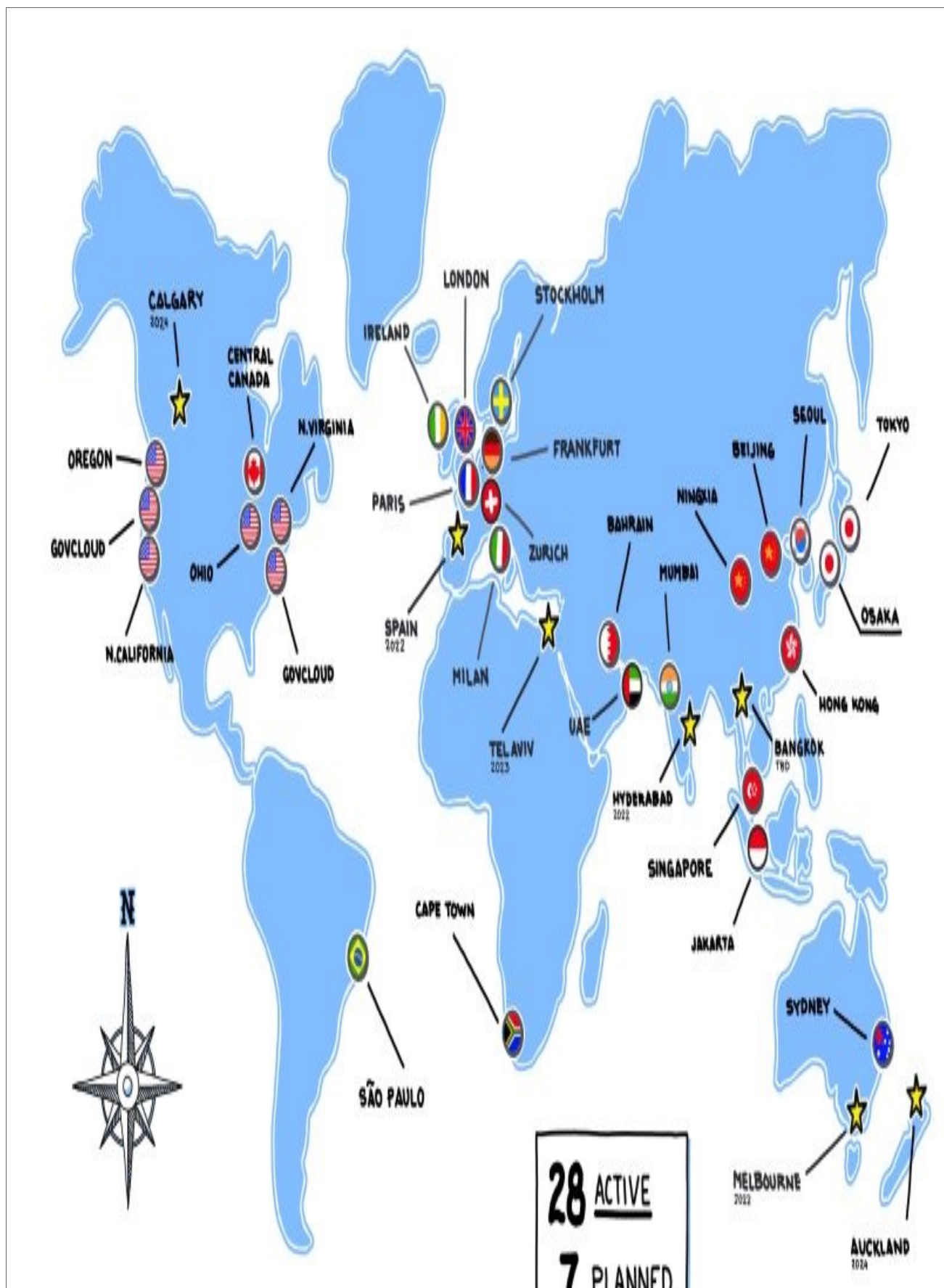
Regions



Availability
Zones



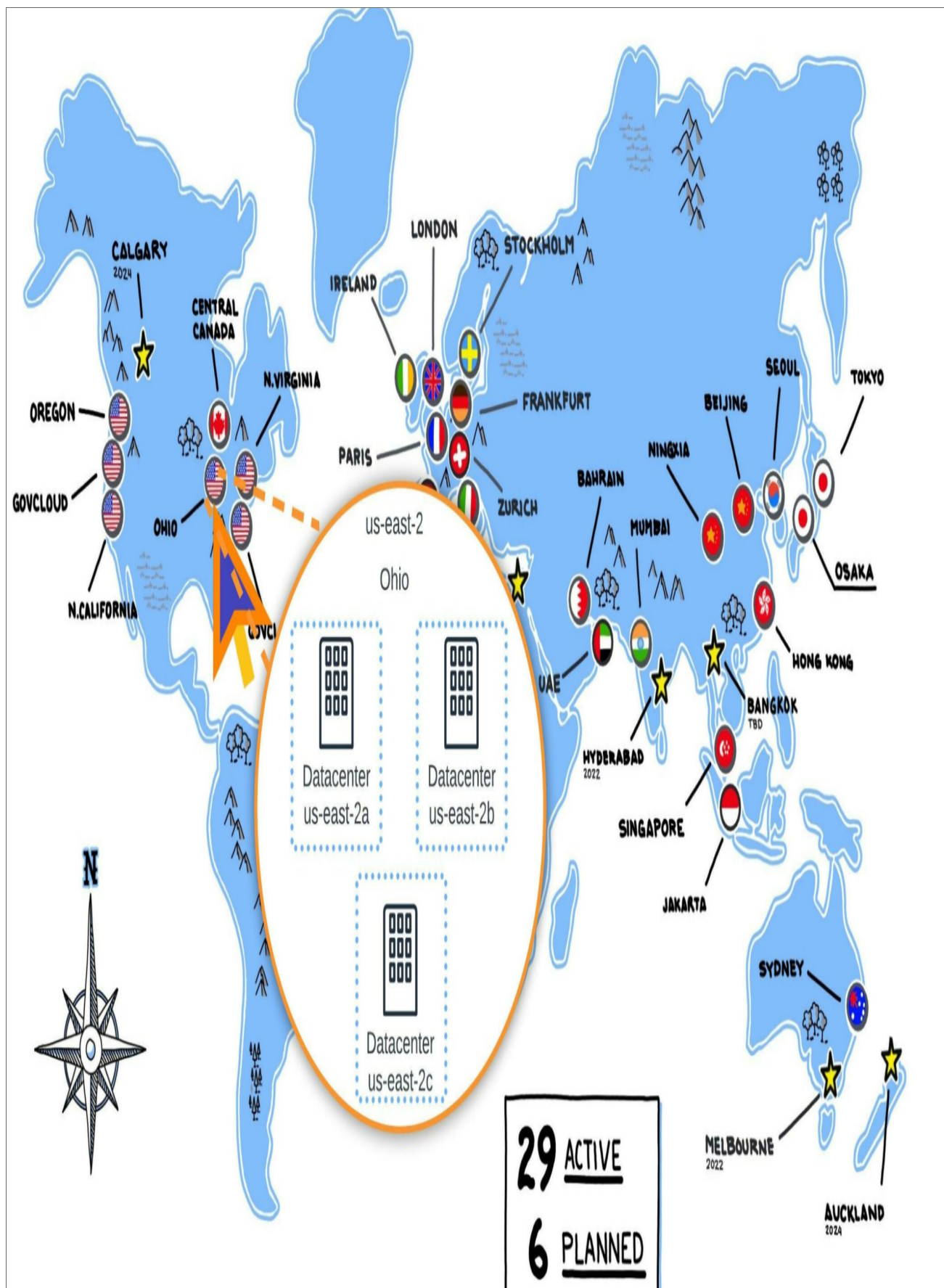
Edge Locations



AWS REGIONS

 @awsgeek
11/11/22

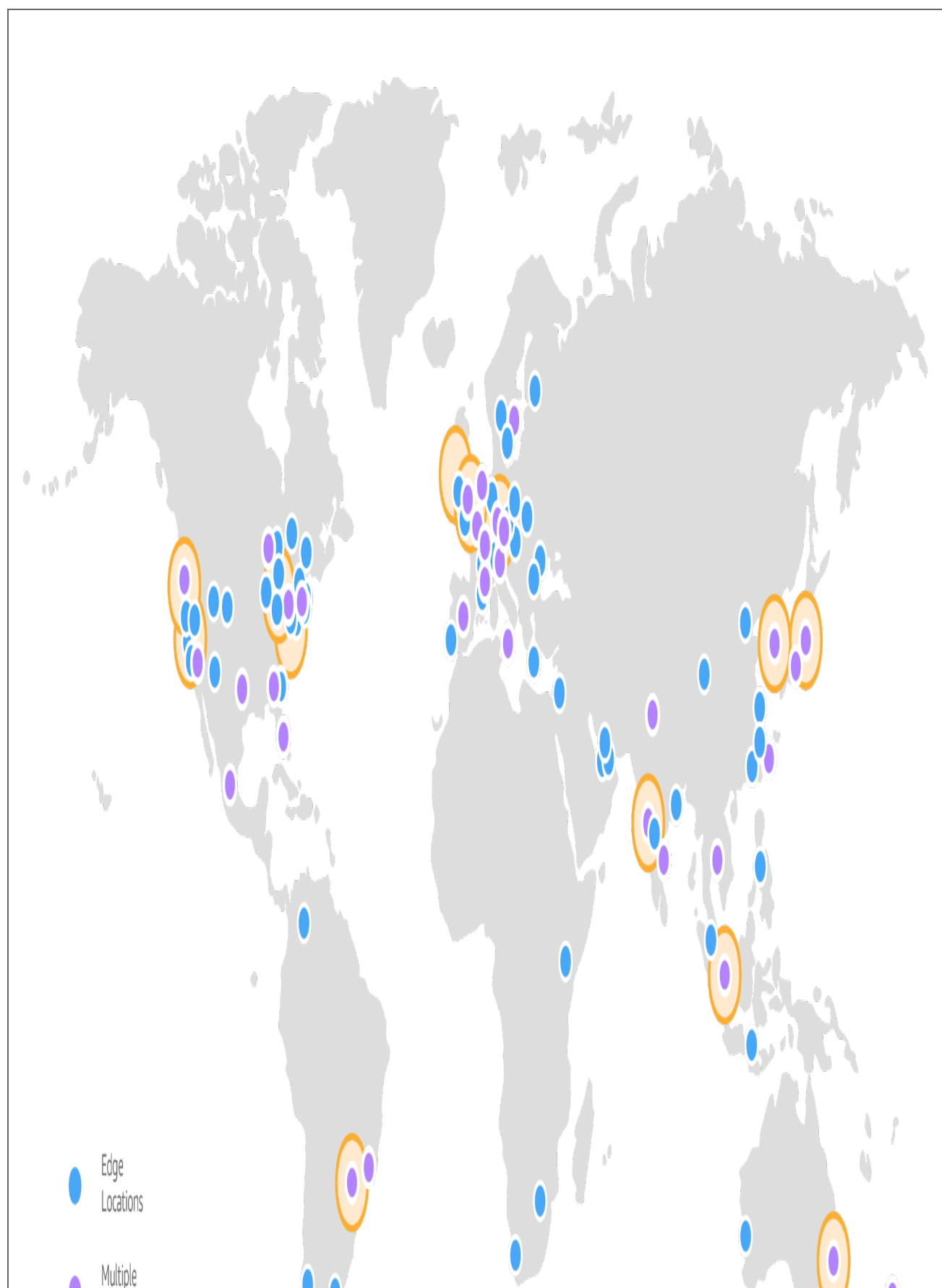


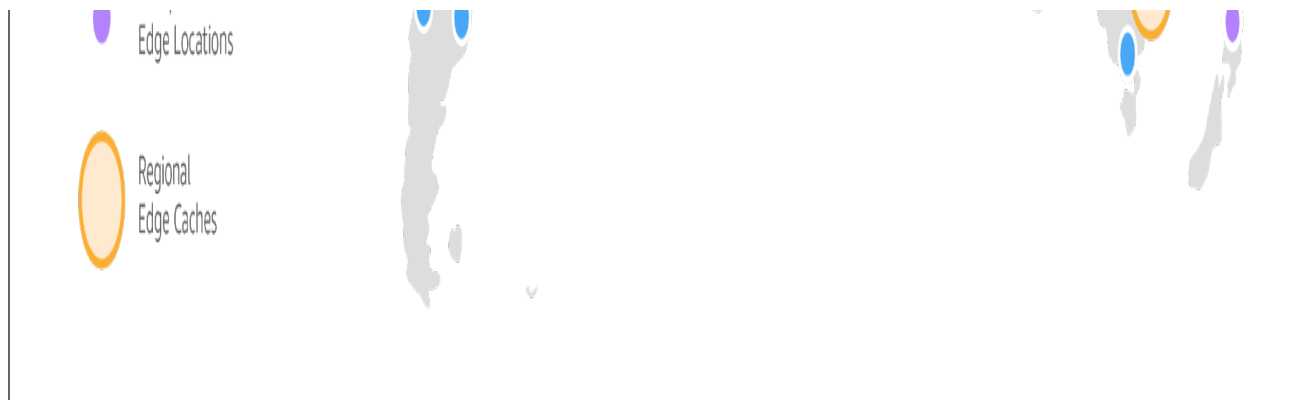


AWS REGIONS

 @awsgeek
11/16/22







SERVICIOS DE AWS MÁS UTILIZADOS

1. **Cómputo**
2. **Almacenamiento**
3. **Redes (Networking)**
4. **Bases de Datos**
5. **Gestión y Seguridad**

CÓMPUTO

- **EC2 (Elastic Compute Cloud):**
 - Servidores virtuales (instancias).
 - Es el servicio más básico y antiguo.
 - En IaC, lo primero que se aprende es a levantar una "t2.micro".
- **Lambda:**
 - Funciones sin servidor (Serverless).
 - Ejecutas código sin gestionar el sistema operativo.
- **ECS / EKS:**
 - Servicios para contenedores (Docker y Kubernetes).
 - Fundamentales para despliegues modernos.

ALMACENAMIENTO

- **S3 (Simple Storage Service):**
 - Almacenamiento de objetos.
 - Se usa para todo:
 - Guardar imágenes
 - Alojar sitios web estáticos
 - Guardar los archivos de "estado" (*state files*) de Terraform.
- **Amazon Elastic File System (Amazon EFS):**
 - Es un servicio de almacenamiento de archivos compartido, elástico y sin servidor (serverless).
 - Permite que múltiples instancias (como Amazon EC2, ECS, EKS o AWS Lambda) monten y accedan al mismo sistema de archivos de forma simultánea.

REDES (NETWORKING)

- **VPC (Virtual Private Cloud):**
 - Tu red privada lógica.
 - Aquí defines subredes, tablas de rutas y seguridad.
 - **Es el corazón de la IaC**, ya que nada funciona sin una red bien definida.
- **Security groups:**
 - Actúan como cortafuegos virtuales a nivel de VM para
 - Controlan el tráfico de red entrante y saliente de tus recursos en la nube
- **Route 53:**
 - El servicio de DNS para gestionar dominios y tráfico.

BASES DE DATOS

- **RDS (Relational Database Service):**
 - Bases de datos gestionadas (MySQL, PostgreSQL, etc.).
- **DynamoDB:**
 - Base de datos NoSQL de altísimo rendimiento y escalabilidad.

GESTIÓN Y SEGURIDAD

- **IAM (Identity and Access Management):**
 - Gestión de usuarios y permisos.
 - **Vital en IaC:**
 - Si el código no tiene los permisos adecuados, no podrá crear recursos.
- **CloudWatch:**
 - Monitorización y logs.

QUÉ ES LO QUE HEMOS APRENDIDO

1. Identificar los diferentes modelos de servicio y despliegue de la nube.
2. Reconocer los componentes básicos de infraestructura (cómputo, almacenamiento, red) que se pueden gestionar en un entorno cloud.
3. Comprender el concepto de responsabilidad compartida y su implicación en la gestión de la infraestructura.

Speaker notes